

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Wed 30 JANUARY 2019 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

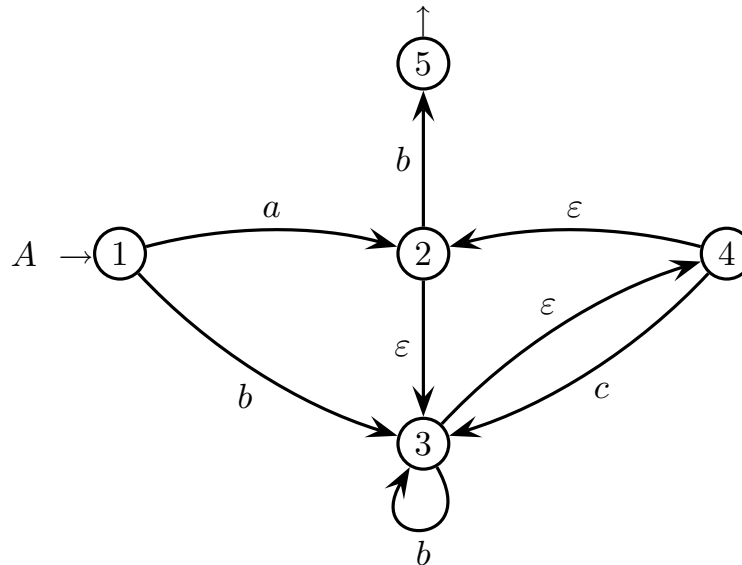
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic finite-state automaton A below, over a three-letter alphabet $\Sigma = \{a, b, c\}$:



Answer the following questions:

- (a) Examine automaton A and list all the accepted strings of length from 0 up to 3 (included). Say if automaton A is ambiguous and explain why yes or no.
- (b) Cut all the spontaneous transitions (ε -transitions) of automaton A and obtain an equivalent automaton A' , without spontaneous transitions, no matter if still nondeterministic.
- (c) By using the Berry-Sethi (BS) method, obtain a deterministic automaton A'' equivalent to automaton A . You may choose to start from automaton A or A' . Say if automaton A'' is minimal (explain why yes or no) and minimize it, if necessary.
- (d) Test that the automaton A'' obtained at point (c) is correct, by verifying it with the short strings found at point (a).
- (e) (optional) Determine whether language $L(A)$ is local and explain why yes or no.

2 Free Grammars and Pushdown Automata 20%

1. The following grammar G , over a two-letter terminal alphabet $\{a, b\}$, generates all and only the strings that contain the same number of letters a and b (axiom S):

$$G \left\{ \begin{array}{l} S \rightarrow a S b \\ S \rightarrow b S a \\ S \rightarrow S S \\ S \rightarrow \varepsilon \end{array} \right.$$

Answer the following questions:

- (a) Verify that grammar G is ambiguous, by drawing at least two syntax trees for the valid string:

$a b a b$

Find the ambiguity degree of the above string and adequately justify your answer.

- (b) Modify grammar G and obtain a new grammar G' (possibly ambiguous), such that (in addition to the original property that the number of letters a and b is the same over the entire string) for every prefix x of each string $s \in L(G')$, this inequality holds:

$$| |x|_a - |x|_b | \leq 1$$

In words, the modulus (absolute value) of the difference between the numbers of letters a and b in the prefix x is smaller than or equal to 1.

Sample valid strings:

$a b b a \quad b a a b \quad a b a b \quad b a b a$

Sample invalid strings:

$b b a a \quad a a b b$

In modifying grammar G , you may add new nonterminals, but keep in mind that simple solutions are preferred.

- (c) Modify grammar G and obtain a new grammar G'' (possibly ambiguous), such that (in addition to the original property that the number of letters a and b is the same over the entire string) for every prefix x of each string $s \in L(G'')$, this inequality holds:

$$|x|_b \geq |x|_a$$

Sample valid strings:

$b a b a \quad b b a a \quad b b a b a a$

Sample invalid strings:

$b a a b \quad a b b a \quad b a b a a b$

Keep in mind that simple solutions are preferred.

- (d) (optional) If grammar G'' is ambiguous, write an equivalent non-ambiguous grammar G''' .

2. Consider a script language that defines two-dimensioned tables, specified as follows:

- a script defines a series of one or more table definitions, without separator
- a table definition has a title and contents
- the table contents consist of a series of one or more columns, separated by semicolon, and each column has a title and contents
- the column contents are a list of one or more values (as defined below), separated by comma
- a group of consecutive columns (one or more) may be covered (its contents are not visualized) by using the keywords `hide-unhide`; two or more groups of columns, adjacent or separated, may be covered
- a group of consecutive values in a column may be covered (same as before) by using the keywords `mask-unmask`
- a title (table or column) is an identifier; a value can be a number or a subtable
- a subtable is like a table, but it does not have a title (only its columns are titled)

Consider the following sample table, with some covered parts, and its script below:

ExampleTableTitle				
alpha		beta	gamma	delta
5		–	–	7
epsilon	rho	–	–	–
11	13			
–	17			
9		–	–	–

```

table ExampleTableTitle contents are
  column alpha is
    5,
    subtable contents are
      column epsilon is 11, mask 15 unmask endcol;
      column rho      is 13, 17 endcol
    endsubtbl,
    9
  endcol;
hide
  column beta  is 30, 31, 32 endcol;
  column gamma is 40, 41, 42 endcol
unhide;
  column delta is 7, mask 8, 9 unmask endcol
endtbl

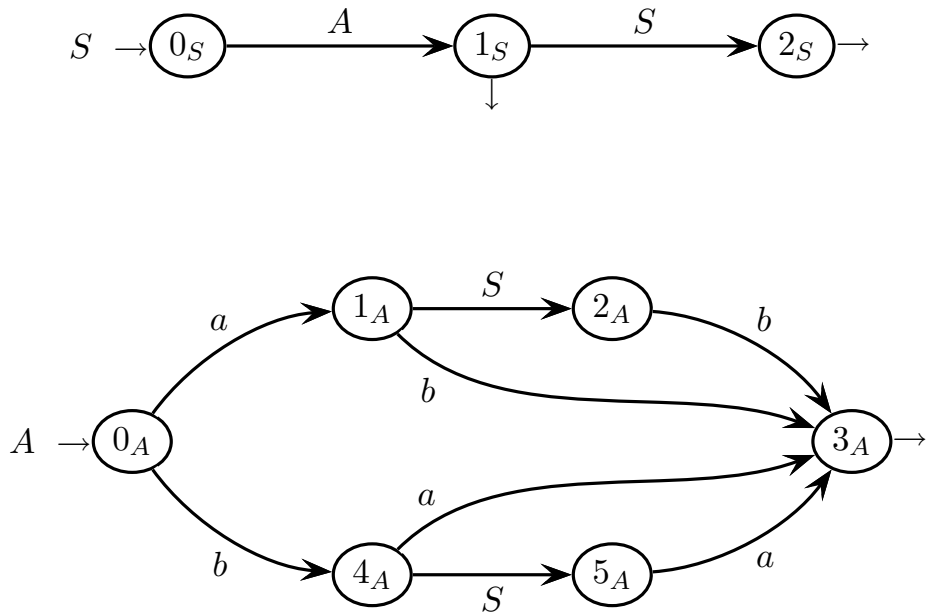
```

You should infer from this sample script any information that is not explicitly given above, in particular the symbols for keywords, delimiters and their placement, etc.

Write a grammar, possibly of type *EBNF* and not ambiguous, that generates the script language described above. *In the grammar, schematize the identifiers and numbers by terminals `id` and `num`, respectively, without expanding them with rules.*

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, represented as a machine net, over a two-letter terminal alphabet $\Sigma = \{ a, b \}$ and a two-symbol nonterminal alphabet $V = \{ S, A \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw a minimal portion of the pilot of grammar G , sufficient to show that grammar G is not $ELR(1)$, and suitably explain why the $ELR(1)$ property fails.
- (b) Write all the guide sets of the machine net of grammar G (on the call arcs and exit arrows), and discuss whether grammar G is $ELL(1)$. In the case it is not so, list all the guide set conflicts that are present.
- (c) By using the Earley method, analyze the following valid string:

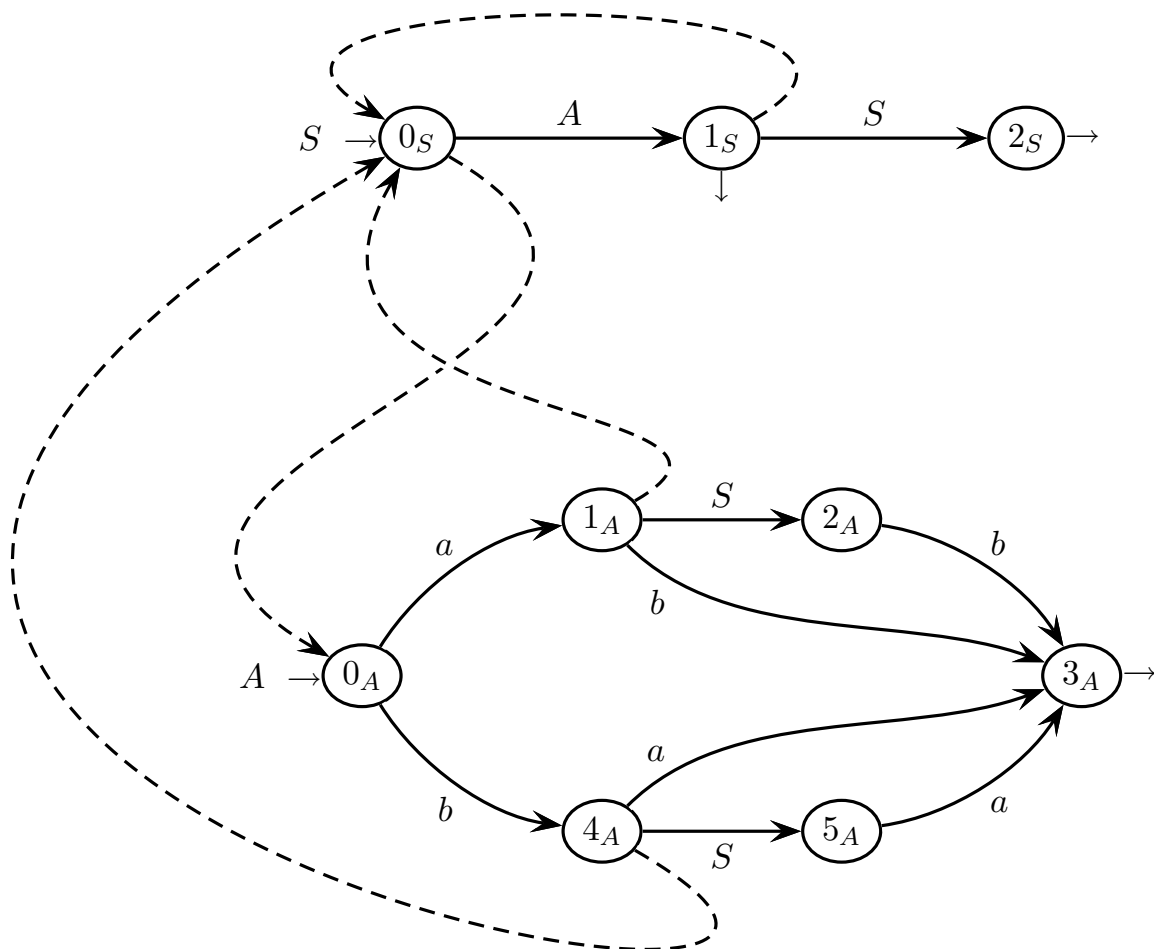
$$a b a b \in L(G)$$

On the Earley vector, highlight the item(s) that determine the acceptance of the above string and possibly of any string prefix.

- (d) (optional) Draw any possible syntax tree of the string analyzed at point (c).

question (a) – please draw here the (minimal part of the) pilot of grammar G

question (b) – please write here all the guide sets on the call arcs and exit arrows



question (c) – please analyze here string $a b a b \in L(G)$

Earley vector of string $a b a b$ (to be filled in – the number of rows is not significant)

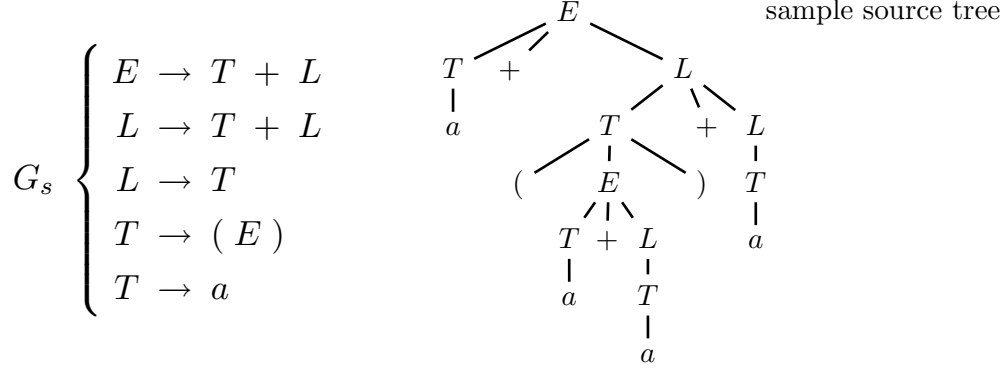
[illegible]

question (d) – please draw here any possible syntax tree of string $a b a b \in L(G)$

free space for continuations of any questions of any exercise

4 Language Translation and Semantic Analysis 20%

1. Consider the source grammar G_s below, for the arithmetic expressions with infix addition $+$, round brackets (subexpressions), and variables schematized by terminal a (axiom E):



Sample source strings are: $a + a$, $a + a + a$, $a + (a + a) + a$ (syntax tree above right).

Consider a translation that converts addition into prefix form with multiple addends (possibly more than two). The idea is to convert $a + a + a$ into $+3 a a a$, for instance, where number 3 specifies the actual number of addends.

Here we want to design a syntactic translation function τ that produces such a multiple addend prefix form, though it does not compute the number of addends. Instead, it outputs a placeholder schematized by terminal n , e.g., $\tau(a + a + a) = +n a a a$. A semantic translation, *not to be considered here*, will compute the actual number of addends and will substitute such a number to the placeholder n .

See these examples (the actual number of addends is shown below n as a comment):

$$\begin{aligned} \tau(a + a) &= + \underset{2}{n} a a \\ \tau(a + a + a) &= + \underset{3}{n} a a a \\ \tau(a + (a + a) + a) &= + \underset{3}{n} a + \underset{2}{n} a a a \quad - \text{translation } \tau \text{ of the example} \end{aligned}$$

Answer the following questions:

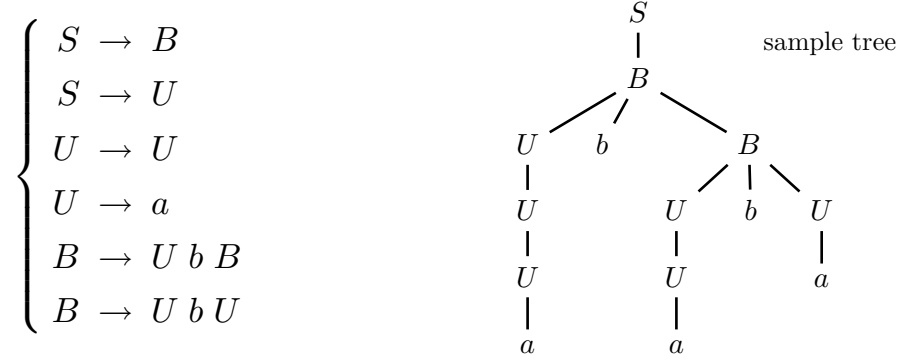
- (a) Write a translation scheme G_τ that defines the syntactic translation function τ described above. Do not change the source grammar G_s . Test scheme G_τ by drawing the syntax translation tree of the sample translation τ above.
- (b) Write a translation scheme $G_{\tau'}$ for an optimized syntactic translation τ' that does not output the placeholder n if addition has only two operands, as follows:

$$\tau'(a + (a + a) + a) = + \underset{3}{n} a + a a a \quad - \text{sample translation } \tau'$$

For τ' you may change the source grammar G_s , if necessary. Test scheme $G_{\tau'}$ by drawing the syntax translation tree of the sample translation τ' above.

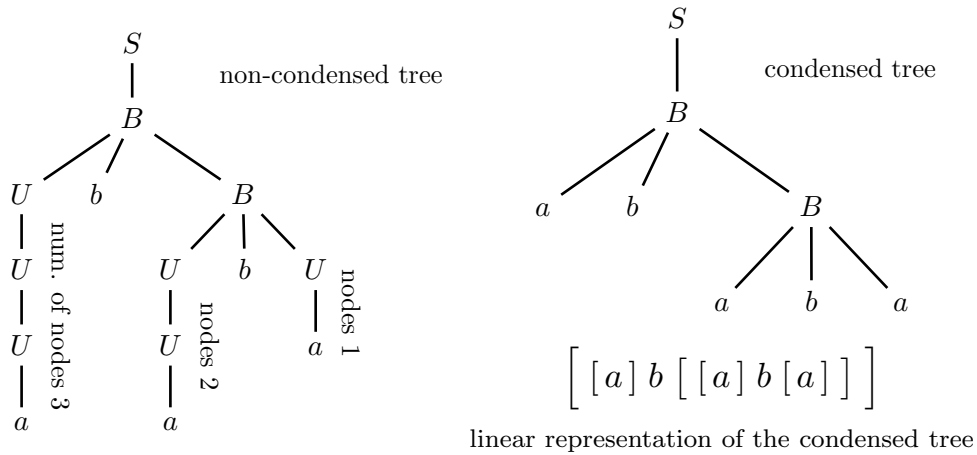
- (c) (optional) Determine whether translations τ and τ' can be computed deterministically by an $ELL(k)$ parser with write actions, for suitable values of $k \geq 1$, and motivate your answers (yes or no).

2. The grammar below (axiom S), over a two-letter terminal alphabet $\{a, b\}$, models the abstract syntax of (simplified) expressions with binary operator b and atomic operand a . The copy rule $U \rightarrow U$ allows the trees of such expressions to have non-branching paths of arbitrary length, as exemplified in the sample tree on the right:



This grammar models an *abstract* syntax, thus it is unsuitable for syntax analysis.

A *condensed* syntax tree is one where the internal nodes of a non-branching path are canceled, as shown below (related to the same sample tree as above):



Answer the following questions (use the tables / drawings / spaces on the next pages):

- Write an attribute grammar that computes, in the tree root, a string constituting the linear (parenthesized) representation of the *condensed* syntax tree. Use only the attribute s specified, and, to compute it, use the concatenation operator “ $\cdot$ ”. Make sure that the grammar is one-sweep and explain why.
- Write an attribute grammar that computes, in the tree root, an *extended* linear representation of the condensed syntax tree, where an integer is inserted next to each substring “ $[a]$ ”. Such an integer indicates the number of consecutive nodes U occurring in the non-branching path immediately above the corresponding leaf node “ a ”. The extended linear representation of the sample tree will thus be “ $[[a]3 b [[a]2 b [a]1]]$ ”. Use both the attributes s and n specified. To insert such integers into the extended linear representation, you may use the function $itos(i)$, which converts an integer i into its string representation.
- (optional) Decorate the sample tree with the values of attributes s and n .

attribute specifications – questions (a) and (b)

<i>name</i>	<i>type</i>	<i>domain</i>	<i>symbol</i>	<i>meaning</i>
<i>s</i>	left	string	S, B, U	the linear representation of the condensed (sub)tree rooted at the current node
<i>n</i>	left	integer ≥ 1	U	the number of consecutive nodes U in a vertical downward path starting from the current node (included); for instance, for the three branches of nodes U in the sample tree (from left to right), attribute n is equal to 3, 2 and 1

free space for continuing answers – questions (a) (b) and (c)

syntax *semantics* – question (a)

1: $S_0 \rightarrow B_1$

2: $S_0 \rightarrow U_1$

3: $U_0 \rightarrow U_1$

4: $U_0 \rightarrow a$

5: $B_0 \rightarrow U_1 \text{ } b \text{ } B_2$

6: $B_0 \rightarrow U_1 \text{ } b \text{ } U_2$

syntax *semantics* – question (b)

1: $S_0 \rightarrow B_1$

2: $S_0 \rightarrow U_1$

3: $U_0 \rightarrow U_1$

4: $U_0 \rightarrow a$

5: $B_0 \rightarrow U_1 \text{ } b \text{ } B_2$

6: $B_0 \rightarrow U_1 \text{ } b \text{ } U_2$

question (c) – decorate here the sample syntax tree with attributes s and n

